Multimodal RAG Chatbot: Detailed Presentation Explanation

Presented by Mansi Suresh Shirode, Volunteer, Gudsky Research Foundation. Mentor: Abhishek Kumar Shukla.

This document explains every slide of the presentation in detail: what is shown, what it means, and what to say. It is written so that someone with no AI background can understand the whole project.

The project in one minute

Normal chatbots answer from memory, so they can guess wrongly and they have never read your own documents. This project builds a chatbot that first looks inside your document (a PDF) and returns the passages that best match your question. That technique is called RAG (Retrieval-Augmented Generation). The project reads a PDF, cuts it into small pieces, converts each piece into numbers that capture its meaning, and stores them in a database called ChromaDB. When you ask a question, the question is converted into numbers the same way, and the database returns the two closest pieces. It is built with Python, LangChain, ChromaDB, HuggingFace embeddings (all-MiniLM-L6-v2), PyPDF and Pillow, and it runs completely on your own computer.

One-line summary: we store your documents in a way a computer can search by meaning, so when you ask a question it fetches the real passages from your files instead of guessing.

Slide 1: Title

On the slide: the Gudsky Research Foundation logo, the foundation name and a short description, the project title "Multimodal RAG Chatbot", the tagline, the mentor (Abhishek Kumar Shukla), your role (Volunteer), your name (Mansi Suresh Shirode), and three boxes on the right: Your documents, Smart search index, Relevant answers.

Explanation: this slide introduces who you are and what the project is. The three boxes on the right are the whole project in three words: documents go in, a smart search index is built, and relevant answers come out. The word "multimodal" means the system is designed for more than one kind of data, here text and images. "RAG" is the technique behind it.

What to say: "I'm Mansi Suresh Shirode, a volunteer at Gudsky Research Foundation, guided by Abhishek Kumar Shukla. My project is a chatbot that answers questions from your own documents. I'll explain it from scratch, so no AI background is needed."

Slide 2: What you will learn

On the slide: six numbered boxes: The problem, The idea (RAG), The big picture, Under the hood, The code step by step, Run it and next steps.

Explanation: this is the agenda. The talk moves from the simple to the technical: first why the project is needed, then the idea that solves it, then the tools and architecture, then the three key concepts, then the real code, and finally how to run it and where it can go next.

What to say: "I'll go in six parts, from the problem to the code. You don't need to know AI to follow."

Slide 3: The problem

On the slide: four short points (Makes things up, Old knowledge, Cannot see your files, Often text only) and a picture comparing a normal chatbot with a RAG chatbot.

Explanation in detail:

- **Makes things up (hallucination):** a chatbot is built to always produce an answer. When it does not know, it can invent something that sounds confident but is wrong.
- **Old knowledge:** a chatbot only knows what it learned during training. Anything newer is missing.
- **Cannot see your files:** your notes, reports and PDFs were never part of its training, so it cannot answer questions about them reliably.
- **Often text only:** real documents contain images, charts and diagrams, which simple systems ignore.

The picture shows the same question asked twice: "What database does our project use for embeddings?" The normal chatbot guesses "PostgreSQL" because it never saw your files. The RAG chatbot answers "ChromaDB" and points to the document it found it in. The picture is an illustration, not a screenshot.

What to say: "If I ask a normal chatbot about my own internship PDF, it has never read it, so it guesses. We need a system that actually reads the document."

Slide 4: The idea, RAG as an open-book exam

On the slide: three steps joined by arrows: Retrieve, Augment, Generate, and a note that this project implements the Retrieve step.

Explanation in detail: think of two students. One answers from memory and may be wrong. The other first finds the correct page in the book and then answers. The second student is more reliable, and RAG works the same way.

- **Retrieve:** find the parts of your documents that match the question.
- **Augment:** hand those parts to the answering step as context.
- **Generate:** write the answer using only that context.

It is important to be clear about scope: your project builds the Retrieve step completely. It finds and shows the best-matching passages. Adding a language model to write a full sentence answer (the Generate step) is future scope.

What to say: "RAG means looking up first and answering second. My project builds the first part, the retrieval engine."

Slide 5: The project in one picture

On the slide: a four-box flow (You give, The system, You ask, You get) and a Goals box.

Explanation in detail: the flow describes the user's view of the project. You give a PDF. The system reads it, splits it and stores it as searchable data. You ask a question in plain English. You get back the most relevant passages from your document.

The three goals are:

1. Answer from your own documents instead of guessing.
2. Run locally with free, open-source tools, with no paid API.
3. Prepare the ground for text plus image (multimodal) search.

What to say: "Documents in, question in, relevant passages out. Everything runs on a normal laptop."

Slide 6: Tools used, in plain English

On the slide: six cards, one per tool.

Explanation in detail: each tool has exactly one job.

- **Python:** the programming language the project is written in.
- **LangChain:** ready-made parts for loading documents and splitting text, so nothing is written from scratch.
- **ChromaDB:** the vector database that stores the searchable data on disk.
- **HuggingFace MiniLM (all-MiniLM-L6-v2):** the model that turns sentences into numbers that capture their meaning. Each piece of text becomes a list of 384 numbers.
- **PyPDF:** reads the text inside PDF files.
- **Pillow:** opens and checks image files.

All of these are free and run locally, so your documents never leave your computer and there are no API costs.

What to say: "Every tool is free and runs locally. Each does one job, and together they make the pipeline."

Slide 7: How the parts connect

On the slide: two rows of boxes. Step A: PDF in data folder, Split into small chunks, Convert to vectors, Save in ChromaDB. Step B: You type a question, Convert to a vector, Find closest chunks, Show top 2 passages.

Explanation in detail:

- **Step A, build the knowledge base (app.py, run once):** the PDF is read, cut into small chunks, each chunk is converted into numbers, and everything is saved in ChromaDB on disk.
- **Step B, ask questions (chat.py, any time):** your question is converted into numbers, the database finds the chunks closest to it, and the top 2 passages are shown.

The key point is that both steps use the same embedding model and the same database. Only then can a question and a document chunk be compared fairly, like measuring two distances with the same ruler.

What to say: "Step A happens once. Step B repeats for every question. Both use the same model, which is why the comparison works."

Slide 8: Three ideas that make it work

On the slide: Chunk, Embedding and Vector database, each with an analogy, and a map showing similar sentences close together.

Explanation in detail:

- **Chunk:** a small piece of text, like one paragraph. Cutting a long document into chunks lets the system return the exact paragraph instead of a whole book. Chunks overlap by 50 characters so that a sentence cut at the edge still appears whole in the neighbouring chunk.
- **Embedding:** meaning turned into numbers, like GPS coordinates for meaning. "Which database stores vectors?" and "Where are embeddings kept?" use different words, but they get nearby numbers because they mean almost the same thing.
- **Vector database:** a library catalogue that finds text by meaning, not by exact words. A normal keyword search would miss a match when the words differ. A vector search finds it because the meaning is close.

The map on the right is a simplified two-dimensional picture of this idea: related sentences cluster together, while unrelated ones (lunch menu, cricket score) sit far away. Real embeddings have 384 numbers, not two, so the map is only an illustration.

What to say: "This is the key slide. Chunks are paragraphs, embeddings are meaning as numbers, and the vector database searches by meaning." Go slowly here.

Slide 9: Step A in detail (app.py)

On the slide: four numbered steps and an image of the key code lines from app.py.

Explanation in detail:

1. **Check the file:** the code looks for data/sample.pdf and stops with a clear error message if it is missing.
2. **Load the PDF:** PyPDFLoader reads every page of the document.
3. **Split into chunks:** RecursiveCharacterTextSplitter cuts the text into pieces of 500 characters with an overlap of 50.
4. **Embed and save:** HuggingFaceEmbeddings with all-MiniLM-L6-v2 creates the vectors, and Chroma.from_documents saves them in the db folder, so this work is never repeated for every question.

The key lines shown on the slide (line breaks added for readability) are:

```
loader = PyPDFLoader(pdf_path)
documents = loader.load()

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=50)
chunks = text_splitter.split_documents(documents)

embeddings = HuggingFaceEmbeddings(
    model_name="all-MiniLM-L6-v2")
vector_store = Chroma.from_documents(
    chunks, embeddings,
    persist_directory="./db")
```

What to say: "Four lines of thinking: check, load, split, embed and save. The overlap makes sure no sentence loses its meaning at a cut."

Slide 10: Step B in detail (chat.py)

On the slide: four step boxes (Load database, You ask, Find top 2 chunks, Show context), an image of the key code lines from chat.py, and an illustrative terminal output.

Explanation in detail:

1. **Load database:** chat.py opens the db folder. If it does not exist, it tells you to run app.py first.
2. **You ask:** you type a question, for example "What database is used for storing embeddings?" Typing exit quits the chat.
3. **Find top 2 chunks:** similarity_search with k=2 returns the two chunks closest in meaning to your question.
4. **Show context:** the passages are printed, or a message says that no relevant context was found.

The key lines shown on the slide (line breaks added for readability) are:

```
vector_store = Chroma(
    persist_directory=db_path,
    embedding_function=embeddings)

query = input(
    "Ask a question (or type 'exit' to quit): ")

results = vector_store.similarity_search(
    query, k=2)

for i, doc in enumerate(results):
    print(f"[{i+1}] {doc.page_content}")
```

The terminal picture on the slide is an illustration of the output format, with placeholders for the passages. query.py does the same job for one fixed question, which is useful for quick testing, while chat.py loops so you can keep asking.

What to say: "The question is converted to numbers with the same model, the database finds the two closest chunks, and I print them."

Slide 11: The multimodal part (multimodal.py)

On the slide: a four-step image flow (Image file, Pillow opens it, Reads format, size, mode, Check passed), the key code lines, an example picture, and a Next step box.

Explanation in detail: multimodal means handling more than just text. The file multimodal.py loads data/sample.jpg with Pillow and prints its format, size and mode. If the image is missing, it shows a clear message. This proves that the project can work with images next to PDFs.

The key lines shown on the slide (line breaks added for readability) are:

```
img = Image.open(image_path)
print(f"Image successfully loaded! "
      f"Format: {img.format}, "
      f"Size: {img.size}, Mode: {img.mode}")
```

Be honest about the current limit: today the module loads and validates an image. It does not yet search inside images. The next step is to turn image content into text (for example with captioning or OCR) so that a question can also find a picture. The example picture on the slide is an illustration, not your file.

What to say: "This is the foundation for multimodal search. Today it checks images; making them searchable is my next step."

Slide 12: Project files and how to run

On the slide: a file list and three commands.

Explanation in detail:

- **app.py:** builds the knowledge base.
- **chat.py:** the interactive chat.
- **query.py:** tests one question.
- **multimodal.py:** checks images.
- **make_pdf.py:** helper for sample data.
- **data/ and db/:** the input files and the saved index.

To run the project:

1. Install: pip install -r requirements.txt
2. Build the knowledge base: python app.py
3. Chat: python chat.py

Put your PDF in the data folder first, then run the commands in order. Anyone can reproduce the project with these three commands.

What to say: "Three commands reproduce the whole project: install, build, chat."

Slide 13: Learnings and future scope

On the slide: What I learned, Future scope, and a thank-you line.

Explanation in detail:

- **What I learned:** how RAG grounds answers in documents, how text becomes searchable vectors, and how to build an AI pipeline from start to end.
- **Future scope:** add a language model to write full answers, make images searchable by content, support many PDFs and show the source of each result, and build a simple web app so non-programmers can use it.

What to say: "Thank you to my mentor Abhishek Kumar Shukla and to Gudsky Research Foundation for this opportunity. I'm happy to take questions."

Glossary

- **RAG:** Retrieval-Augmented Generation. Look up relevant information first, then answer from it.
- **Hallucination:** when a chatbot invents a confident but wrong answer.
- **Chunk:** a small piece of a document, here about 500 characters.
- **Overlap:** the characters shared between neighbouring chunks (50 here) so no sentence is cut.
- **Embedding / vector:** a list of numbers that captures the meaning of a text. MiniLM produces 384 numbers per chunk.
- **Vector database (ChromaDB):** a database that stores vectors and finds the closest ones to a query.
- **Similarity search:** finding the stored chunks whose vectors are closest to the question's vector.
- **Multimodal:** handling more than one type of data, such as text and images.

Current limitations

Saying these openly shows that you understand the project.

- It returns relevant passages, not a written sentence answer, because there is no language model in the Generate step yet.
- It always returns the 2 closest chunks, even if neither is very relevant, because there is no relevance cutoff.
- It reads one specific file, data/sample.pdf.
- Images are loaded and checked, but not yet searched.

Questions you may be asked

- **How is this different from ChatGPT?** ChatGPT answers from what it learned in training. This project answers from the documents you give it, so each answer can be traced back to its source.
- **Why not just use Ctrl+F?** Ctrl+F needs the exact words. This project finds matches by meaning, even when the words differ.
- **Does it write full answers?** Not yet. It returns the most relevant passages, and adding a language model is the next step.

- **Does my data leave my computer?** No. The model, the database and the code all run locally.
- **Why are there chunks and overlap?** Chunks let it return the exact paragraph, and overlap keeps sentences from being cut at the edges.